

Day 6 Contest: GP of North Beach

Bytedance Moscow Workshops Camp 2020

May 55, 2020

A. Triangle Tiling

In any solution, consider all rhombi with a horizontal side. Connect them into chains by gluing touching horizontal sides.

The chains connect bottom segments of the big triangle with bottom segments of the removed unit triangles.

The border of the uncovered part does not have any horizontal sides, thus we can cover it greedily with rhombuses 2 (see the picture in the statement).

We now only need to produce vertical chains. Consider the following process:

- initially a token is placed in each upward unit triangle in the bottom row 0;
- a token can be moved from any triangle to an adjacent upward triangle higher than before;
- no triangle can be ever visited by more than two tokens;
- if a token arrives in a removed triangle, it stays there forever.

The goal is to place all tokens into removed triangles.

Let $\Delta_{l,r}$ be the equilateral triangle with its base on the bottom of the large triangle covering unit segments $[l, r]$.

At any time, let $T_{l,r}$ and $R_{l,r}$ be the number of Tokens and Removed triangles in $\Delta_{l,r}$ respectively.

If at any intermediate moment we have $T_{l,r} < R_{l,r}$, we cannot satisfy all removed triangles inside $\Delta_{l,r}$.

If we have $T_{l,r} < R_{l,r}$ for some l, r in the beginning, there is no answer.

Otherwise the answer always exists. To show this, let's figure out how to move tokens from the row i to the row $i + 1$ while preserving all conditions $T_{l,r} \geq R_{l,r}$.

Put $D_{l,r} = T_{l,r} - R_{l,r}$.

Suppose there are currently some tokens in the row i , all removed triangles below row i are satisfied, and all $D_{l,r} \geq 0$.

Tokens in removed triangles in row i are caught. All triangles in the row i are satisfied because $D_{l,r} \geq 0$.

Suppose we move a token in the triangle (i, j) (i -th row, j -th triangle from the left) to $(i - 1, j - 1)$. This results in decreasing $D_{j-i, j+i}, \dots, D_{j-i, n}$ by one.

Move from (i, j) to $(i - 1, j)$ decrements $D_{1, j+i}, \dots, D_{j-i, j+i}$.

We can always choose suitable directions for each token. Otherwise, we have a token at (i, x) so that for some l, r currently $D_{l, x+i} = D_{x-i, r} = 0$.

But then at the same moment $D_{l,r} \leq D_{l, x+i} + D_{r, x-i} - 1 < 0$, a contradiction.

Hence it is always possible to move tokens between rows, and solution exists.

Let's move each token (i, j) to $(i - 1, j)$, unless there is a position $l \leq j - i$ such that $D_{l, j+i} = 0$.

Let's maintain values of $D_{l,r}$ in separate segment trees indexed by l for each r .

Moving from (i, j) to $(i - 1, j)$ decrements a segment in $D_{l, j+i}$.

If $\min_{l \leq j-i} D_{l, j+i} = 0$, then the token will always move from (i, j) to $(i - 1, j)$ until it is caught in the closest removed triangle. For each event of this kind we need to make range decrements in $O(n)$ segment trees. At most $O(n)$ of these events will happen.

The total complexity is $O(n^2 \log n)$.

B. Balanced Binary String

We will naturally extend the definition of a *balanced sequence* to arbitrary integer sequences.

Proposition. Let $0 \leq k \leq n$. A balanced string of length n with k 1's exists. All balanced strings of length n with k 1's are cyclic shifts of each other.

Proof. Case $k = 0$ is obvious.

If $k > n/2$, replace 0 with 1 and vice versa, and the case is symmetrical to $k' = n - k$.

If $k \leq n/2$ the string can not have two consecutive 1's.

Cyclically shift the string so that it starts with 1, and separate the string into blocks of the kind 10^* (any number of zeros). Let $a_1, \dots, a_k$ be the sizes of the blocks.

For each i, j $|a_i - a_j| \leq 1$. Otherwise, if, say, $a_i > a_j + 1$, then the block j together with the next 1 have two 1's, and the last $a_j + 1$ digits of the block i have no 1's, a contradiction.

Further, the sequence $a_1, \dots, a_k$ is balanced. The proof is similar to the argument above.

All a_i are either $\lfloor n/k \rfloor$ or $\lfloor n/k \rfloor - 1$, occuring $n \bmod k$ and $k - n \bmod k$ times respectively.

Subtracting $\lfloor n/k \rfloor - 1$ from all a_i we obtain a balanced binary string of length k with $n \bmod k$ 1's. By induction hypothesis it is unique up to a cyclic shift.

The way to restore the original string is also unique. $\square$

Upon closer examination we find that the unique string $s(n, k)$ has k 1's in positions $\lfloor i \cdot n/k \rfloor$ for $i = 0, \dots, k - 1$ (up to a cyclic shift).

Now, for each $s(n, k)$ we need to find the number of its distinct cyclic shifts matching with a pattern. This can be done in $O(n^2/32)$ with bitsets, or in $O(n \log n)$ with FFT.

Note that $s(n, k)$ has period $\text{GCD}(n, k)$.

The total complexity is $O(n^3/32)$ or $O(n^2 \log n)$ (the latter is probably slower in practice).

C. Digital Root

Given a string representing a B -based number, answer several queries of a kind "how many substrings of a string may have digital root x if it is possible to replace at most one digit with some digit in set A ".

The digital root of a number $s_1 s_2 \dots s_n$ in base B equals to $(s_1 + \dots + s_n) \bmod (B - 1)$ unless all $s_i = 0$, in which case the digital root is zero.

Let the number in question be nonzero. We calculate the number of substrings with sum (hereinafter sum is modulo $B - 1$) s and with the set of digits in this substring denoted with mask m .

Let p_{ic} be the number of prefixes of s up to the i -th with sum c . Consider the i -th prefix. Let it have sum c . Which masks can have the substring ending at i ? Write the position of the last occurrence of all digits (up to this prefix) in decreasing order and take the prefix of this list. Let the last taken occurrence be r and the next one l . The mask of all substrings starting between l and r and ending at i is the same. There can be at most B distinct masks, we account them separately.

For each such mask, fix the expected substring sum k . Now the number of substrings with the given mask with sum k ending at i equals to $p_{r, c-k} - p_{l-1, c-k}$.

When we know the number of substrings with given sum and exactly given mask, we run subset convolution algorithm. Now for sum c and mask m we know the number of substrings with sum c consisting only of digits from m (but not necessarily having them all).

To answer the query, we iterate through the current sum of the substring. Given the current sum, the expected sum, and the set of digits used for replacement, we can find the set of digits that should be in the substring to make the replacement and use this mask to lookup the number in the precomputed table.

Queries with zero digital root are special: we cannot make calculations modulo $B - 1$ since the only string with zero digital root is the zero string. To answer such queries we should precalculate the number of substrings with at most one non-zero digit. It is done by considering maximal substrings of consecutive zeroes (such substring of length l adds $l(l+1)/2$ to their number) and maximal substrings of consecutive zeroes to the left and to the right of each nonzero digit (such substrings of l and r zeroes add $(l+1)(r+1)$ to their number).

D. Ternary String Revolution

There is a number of possible substring substitutions (e.g. replace any “00” with “12” and vice versa). We call two strings equivalent if we can get one of them from another with such substitution. We are given a string and several queries of a kind “how many substrings of the original string are equivalent to the string in query?”

The substitutions in the statement define a free group. It can be shown that the group is finite; precisely, it contains 24 elements. To find the group, we may consider all substrings of small length (10 should be more than enough) and apply all possible reductions, taking the shortest string as a representative in an equivalence class (lexicographically smallest in case of tie).

When all the representatives are found, build the Cayley table of the group (it is the multiplication table which shows the equivalence class of the concatenation of two strings of given classes). Also find an inverse element for each class (inverse for string s is such a string s' that ss' is equivalent to an empty string).

Now we know the equivalence class of strings “0”, “1”, “2”. Using the Cayley table we can find the equivalence class of any string.

To solve the problem, we should count the equivalence classes of all substrings of the input string. It is achieved with cumulative sum array. Consider characters one by one and store the number of shorter prefixes that correspond to each equivalence class. Of course, the class of the current prefix should also be stored. Using this information and the inverse array we can save the number of substrings that end at the current character and have certain equivalence classes.

To answer the query, we calculate its equivalence class and lookup the number of its occurrences among substrings.

E. Chiaki Chain Counting

A Chiaki chain is a chain graph with several chains attached to its vertices, each chain ending with a cycle (see problem statement for a precise definition). Count the number of labeled Chiaki chains with given number of vertices and subchains.

Trivial conditions on the number of vertices and edges should apply. The Chiaki chain of order k contains $n + k - 1$ edges and at least $n + 3 + 4 + \dots + k = n + k(k + 5)/2$ vertices. So for the Chiaki chain to exist, $m = n + k - 1$ and $n \geq k(k + 5)/2$ must hold.

We denote with s the value $m - k(k + 5)/2$, which is the number of edges not involved into any cycles. We count the number of different tree topologies. For each topology, the number of labelings is $n!/2^k/iso$, where iso is the number of isomorphisms of the given topology and 2^k comes from the fact that each Chiaki cycle is counted twice, once for each direction.

There are several cases.

- $k = 1$. For $s \geq 2$, there is a single topology with subchain at the end. For $s \geq 3$, there are $\binom{s-1}{2/2}$ additional topologies with subchain in the middle.

Explanation is similar to counting the number of ordered partitionings.

- $k = 2$. For $s \geq 3$, there is a single topology with subchains at the ends. For $s \geq 4$, there are $s(s - 3)/2$ topologies where one subchain is at the end and another is in the middle. Finally, for $s \geq 5$, there are $\binom{s-1}{4}$ topologies where both subchains are in the middle.
- $k = 3$. We will not write explicit formulas here, only topology kinds.
 - center vertex, three subchains going from it, each of them having length at least 2 (original line cannot be determined uniquely);
 - center vertex, three subchains going from it, one having length 1;
 - two subchains at the same end, both having length at least 2, other subchain in the middle;
 - two subchains at the same end, one having length 1, other subchain in the middle;
 - general case — all subchains in the middle.
- $k > 3$.

- Cases are similar to those with $k = 3$. Things to consider: how many open ends does the original line have; for how many ends it can be distinguished which subchain is connected to the end.

F. Subset Sum

Given at most 20 000 positive integers of value at most 20 000, find the maximum possible subset sum not exceeding c .

Use subset sum through balancing algorithm:

<https://chaoxuprime.com/posts/2018-12-18-subset-sum-through-balancing.html>.

G. Back and Forth

You are given a directed graph and two vertices s and t . The cost of visiting the i -th vertex is p_i . Find a way from s to t and back with least possible total cost of visited vertices. The cost of the vertex is counted once even if it is visited several times.

First, calculate the shortest paths between each pair of vertices, where the length of the path is the total cost of all vertices on the path, including the last, excluding the first. Then we run the dynamic programming dp_{xy} : what is the minimum cost of going from y to x and visiting s in between.

The DP is calculated in a Dijkstra-like approach. We consider all pairs (x, y) in order of nondecreasing dp_{xy} using a priority queue. Initially the queue contains only pair (s, s) with value $cost_s$. For each (x, y) , there are three options:

1. For each edge (x, v) , relax the value dp_{vy} with dp_{xy} . If $v \neq y$, the cost of v should be added. This stands for adding vertex v to the end of the path.
2. For each edge (v, y) , relax the value dp_{xv} with dp_{xy} . If $v \neq x$, the cost of v should be added. This stands for adding vertex v to the beginning of the path.
3. Relax the value dp_{yx} with dp_{xy} plus shortest path from x to y minus the cost of y . This stands for reversing the path.

At the end the answer will be at dp_{tt} .

Complexity: $O(n^3 \log n)$.

H. Stone Game

There are two piles of stones. Two players in turn choose a pile with even number of stones and move half of the stones to the other pile. If a previously reached configuration occurs, the game ends with a draw; if a player has no valid move he loses. How many starting configurations result in wins of the first player, wins of the second player, and draws, if there are at most n stones in the first pile and at most m in the second? n and m are given in binary.

Consider the number of trailing zeroes in the sizes of piles in binary. There are three cases:

- if both are equal and even, second player wins;
- if the smallest is odd, first player wins;
- otherwise the game ends with a draw.

The easiest way to find the rule is to run the simulation for small pile sizes.

Now we compute the number of integers less than n with exactly k trailing zeroes for each k (same for integers less than m). It is easier to compute the number of integers with at least k trailing zeroes: their number is $\lfloor n/2^k \rfloor$, which is exactly the prefix of the input. The number of integers with exactly k trailing zeroes is the number of integers with at least k t.z. minus the number of integers with at least $k - 1$ t.z.

Given those arrays, the number of winning/losing/draw positions is calculated by definition using cumulative sum arrays.

I. Longest Lyndon Prefix

For each suffix of a string find its longest prefix that is lexicographically smaller than all of its proper suffixes.

Build a suffix array to compare suffixes of the string in constant time. Now for each suffix i we want to find the first suffix that is shorter than i -th and is lexicographically smaller.

We compute this value from short suffixes to the longer ones. Let p_i be the mentioned value (that is, $p_i - i$ is the answer to the problem for the i -th suffix). It is known that all suffixes between i and p_i are lexicographically greater than the i -th.

For each i , initially set p_i to $i + 1$. Then, while p_i -th suffix is still greater than i -th suffix, move p_i to p_{p_i} skipping the suffixes that are known to be greater.

Analyzing the runtime is left as an exercise to the reader. Another way with simpler analysis is to use segment trees or stack to find the first position on the suffix with the value less than the given one.

J. Program Optimization

You are given a permutation of $1, \dots, n$. Answer several queries of two kinds:

- swap two adjacent elements;
- find $MEX(a_l, \dots, a_r)$.

MEX , or Minimum-Exclusive, of a set is a smallest natural number not present in the set. Since all elements are distinct, the MEX of a subarray is the minimum value outside the subarray.

We keep two arrays, $pref_i = \min(a_1, \dots, a_i)$, and $suf_i = \min(a_i, \dots, a_n)$. When two adjacent elements are swapped, only two elements are changed in $pref$ and suf . The answer to the query $MEX(a_l, \dots, a_r)$ is $\min(suf_{l-1}, pref_{r+1})$.

K. Determination

There is an adjacency matrix of a tree. Replace all 1-s with given numbers and all zeroes with the same number x . Find the determinant of this matrix modulo $10^9 + 7$.

Decompose the matrix into sum of three matrices $xJ + T + D$, where J is a matrix consisting of all ones, T is a matrix of tree edges and D is a diagonal matrix. The determinant equals $\sum_{\sigma} \text{sgn}(\sigma) a_{i\sigma_i}$. When counting permutations, we will also “choose” which matrix will the value $a_{i\sigma_i}$ come from.

Assume both $a_{i\sigma_i}$ and $a_{j\sigma_j}$ are taken from the matrix J . We can interchange the values σ_i and σ_j . The resulting permutation will have the same product of values but different sign, so two permutations will cancel each other. So at most one value can be taken from the matrix J .

If we consider the pairs (i, σ_i) as the edges of the graph, there are three possible kinds of edges:

- J -edge, at most one. J -edge may also be the edge of the tree or a loop;
- D -loop;
- T -edge.

All T -edges go in pairs (u, v) and (v, u) except, maybe, for a path from σ_i to i if (i, σ_i) is a J -edge.

The sum of products over such permutations now can be calculated with a tree DP. The parameters are the vertex itself, whether the vertex is already covered by some edge, and whether there is such i in a subtree that (i, σ_i) is a J -edge. To calculate the parity one should switch the sign each time adding the loop, the pair of T -edges and the J -edge.

L. Fraction Reduction

Given a fraction $\frac{a}{b}$, how many operations is necessary to reach zero? There are two possible operations:

- change fraction x to $-\frac{1}{x}$;

- *change fraction x to $x + 1$.*

Let's see what those operations do to numerator and denominator of a fraction $\frac{a}{b}$: the first operation translates (a, b) to $(-b, a)$, and the second operation translates (a, b) to $(a + b, b)$.

It looks somewhat similar to the steps of Euclidean algorithm. Indeed, the following exactly follows the steps of Euclidean algorithm for numbers “ $-a$ ” and “ b ”:

- if $(-a) \% b = 0$, perform the second operation several times and stop;
- otherwise subtract a from b with the second operation until negative fraction $\frac{b-a\%b}{b}$ is reached and move to the fraction $\frac{-b}{b-a\%b}$ with the second operation.

The process takes $O(\log(|a| + |b|))$ time, and the number of steps could be calculated straightforwardly.